

RLHF, DPO e Modelos Determinísticos

Guia prático para o grupo — projeto Proton Flow

Objetivo: explicar de forma simples como LLMs são alinhados com preferências humanas (RLHF e DPO) e como obter respostas determinísticas/reprodutíveis para usar no projeto.

*Nota: onde o pedido citava "FDO", assumimos que se trata de **DPO (Direct Preference Optimization)**, a técnica que costuma ser comparada ao RLHF.*

1. Contexto: as etapas de treino de um LLM

Um LLM passa por fases. O alinhamento (RLHF/DPO) é a última, e serve para o modelo responder do jeito que humanos preferem: útil, seguro e no formato certo.

Fase	O que faz	Dados
1. Pré-treino	Aprende a prever o próximo token em enormes volumes de texto	Texto bruto (web, livros, código)
2. SFT (Supervised Fine-Tuning)	Aprende a seguir instruções imitando exemplos bons	Pares (instrução, resposta ideal)
3. Alinhamento (RLHF ou DPO)	Aprende a preferir respostas melhores às piores	Comparações: resposta A é melhor que B

2. RLHF — Reinforcement Learning from Human Feedback

Aprendizado por reforço a partir de feedback humano. Em vez de dizer ao modelo a resposta certa, humanos **ranqueiam** respostas, e o modelo é otimizado para gerar as mais bem ranqueadas.

Passo a passo

- **Coleta de preferências:** para um mesmo prompt, o modelo gera 2+ respostas; avaliadores humanos escolhem a melhor (e a pior).
- **Modelo de recompensa (Reward Model):** uma rede treinada para dar uma nota (score) a qualquer resposta, imitando o julgamento humano.
- **Otimização por RL (geralmente PPO):** o LLM gera respostas, o reward model dá a nota, e o LLM é atualizado para aumentar a nota.
- **Penalidade KL:** impede que o modelo se afaste demais do modelo SFT original (evita que ele 'trapaceie' o reward model e gere texto estranho).

```
prompt --> LLM (policy) --> resposta --> Reward Model --> nota
          ^                                     |
          +----- atualização PPO (+ penalidade KL) <-+
```

Vantagens e desvantagens

- + Muito eficaz; foi a base do ChatGPT/InstructGPT.
- - Complexo: 3-4 modelos em memória (policy, referência, reward, value).
- - Instável e caro; sensível a hiperparâmetros.
- - Risco de *reward hacking* (otimizar a nota e não a qualidade real).

3. DPO — Direct Preference Optimization

O DPO (Rafailov et al., 2023) alcança o mesmo objetivo do RLHF **sem reward model e sem RL**. Ele usa diretamente os pares de preferência numa função de perda simples, tratada como um problema de classificação.

Dado um prompt x , uma resposta preferida y_w (winner) e uma rejeitada y_l (loser), a perda é:

$$L = -\log \text{sigmoid}(\beta * [\log(\pi(y_w|x)/\text{pref}(y_w|x)) - \log(\pi(y_l|x)/\text{pref}(y_l|x))])$$

π = modelo sendo treinado

pref = modelo de referência (congelado, normalmente o SFT)

β = força da restrição ao modelo de referência (típico: 0.1)

Em palavras: aumentar a probabilidade da resposta boa e diminuir a da ruim, relativamente ao modelo de referência.

Exemplo de dataset (formato comum)

```
{"prompt": "Resuma a reunião em JSON.",
"chosen": "{\"temas\": [\"prazo\"], \"acoes\": [...]}", # resposta boa
"rejected": "Claro! A reunião foi sobre vários assuntos..."}
```

Exemplo de código (biblioteca TRL, Hugging Face)

```
from trl import DPOTrainer, DPOConfig
from datasets import load_dataset

ds = load_dataset("json", data_files="prefs.jsonl")["train"]
cfg = DPOConfig(output_dir="dpo-out", beta=0.1, learning_rate=5e-7,
                 per_device_train_batch_size=2, num_train_epochs=1)
trainer = DPOTrainer(model=model, args=cfg, train_dataset=ds,
                     processing_class=tokenizer)

trainer.train()
```

RLHF x DPO

Critério	RLHF (PPO)	DPO
Reward model	Sim, separado	Não (implícito)
Aprendizado por reforço	Sim	Não
Complexidade / custo	Alta	Baixa
Estabilidade	Sensível	Estável
Dados necessários	Preferências + prompts	Pares chosen/rejected
Quando usar	Grande escala, recompensa programática	Equipes pequenas, projetos como o nosso

Recomendação para o grupo: se for preciso alinhar um modelo, DPO é o caminho mais viável. Mas, para o Proton Flow, que usa um modelo local via Ollama, muitas vezes **prompt + schema +**

determinismo (seção 4) já resolve sem treinar nada.

4. Como fazer um modelo determinístico

Determinístico = a mesma entrada gera sempre a mesma saída. LLMs são probabilísticos: a cada passo escolhem o próximo token por amostragem. Para torná-los (quase) determinísticos, controlamos essa amostragem.

4.1 Parâmetros de decodificação

Parâmetro	Valor p/ determinismo	Efeito
temperature	0	Escolhe sempre o token mais provável (greedy)
top_k	1	Considera só o token mais provável
top_p	1.0 (ou irrelevante com top_k=1)	Sem corte aleatório por probabilidade acumulada
seed	fixo (ex.: 42)	Fixa o gerador de números aleatórios
num_predict / max_tokens	fixo	Limita o tamanho da resposta
repeat_penalty	fixo	Evita variação por penalidades dinâmicas

4.2 Aplicando no projeto (Ollama)

Hoje o backend/analysis_service.py usa temperature: 0.1, o que ainda permite variação. Para determinismo, o payload ficaria assim:

```
payload = {
    "model": self.model_name,
    "prompt": texto_enriquecido,
    "system": SYSTEM_PROMPT_V2,
    "stream": False,
    "format": "json",          # ou um JSON Schema completo
    "options": {
        "num_ctx": OLLAMA_NUM_CTX,
        "num_predict": OLLAMA_NUM_PREDICT,
        "temperature": 0,      # era 0.1
        "top_k": 1,
        "top_p": 1.0,
        "seed": 42,
    },
}
```

4.3 Saída estruturada

- Passe um **JSON Schema** no campo format do Ollama (em vez de só "json") para forçar a estrutura da resposta.
- Valide a saída com **Pydantic** (o projeto já tem schemas.py) e rejeite/refaça se inválida.
- Mantenha um **fallback determinístico** por regras (o projeto já tem analysis_engine = deterministic_fallback).

4.4 Controle de ambiente e versões

- Fixe a **versão exata do modelo** (tag/digest no Ollama, ex.: llama3:8b-instruct-q4_K_M) — atualizar o modelo muda as respostas.

- Fixe o **prompt** (versionado, ex.: SYSTEM_PROMPT_V2) e o pré-processamento do texto de entrada.
- Use a mesma **quantização** e, se possível, o mesmo hardware.

4.5 Limitações importantes

- Mesmo com temperature 0 e seed fixo, GPUs podem gerar pequenas diferenças (operações de ponto flutuante em paralelo, batching). Determinismo é **quase total**, não absoluto.
- Com APIs hospedadas de terceiros, nem sempre há garantia — o modelo pode mudar sem aviso.
- Para garantia 100%, use **cache**: guarde (hash da entrada + versão do modelo + versão do prompt) -> saída, e reutilize.

```
import hashlib, json
def cache_key(texto, modelo, prompt_v):
    return hashlib.sha256(f"{modelo}|{prompt_v}|{texto}".encode()).hexdigest()
```

5. Existe alguma relação entre alinhamento e determinismo?

Sim, indireta. Alinhar (RLHF/DPO) torna o modelo mais **consistente** no formato e no estilo, porque concentra a probabilidade em poucas respostas boas. Mas só os parâmetros de decodificação (seção 4) tornam a saída **reprodutível**. O ideal é combinar: bom prompt/modelo alinhado + decodificação determinística + validação de schema.

6. Checklist para o projeto

- [] Trocar temperature para 0, adicionar top_k=1 e seed.
- [] Usar JSON Schema no format do Ollama.
- [] Fixar versão do modelo e do prompt.
- [] Validar com Pydantic e manter o fallback determinístico.
- [] Implementar cache por hash da entrada.
- [] Criar testes que rodam a mesma reunião N vezes e comparam as saídas.
- [] (Opcional) Se a qualidade ainda não for suficiente, montar um dataset de preferências (chosen/rejected) com as análises corrigidas pela equipe e fazer DPO.

7. Glossário

Termo	Significado
Policy	O modelo que está sendo treinado
Reward model	Modelo que dá nota à qualidade de uma resposta
PPO	Algoritmo de RL usado no RLHF
KL divergence	Medida de quanto o modelo se afastou da referência
SFT	Ajuste fino supervisionado com exemplos
Greedy decoding	Escolher sempre o token mais provável
Seed	Semente do gerador aleatório, para reprodutibilidade

Referências: Ouyang et al., 2022 (InstructGPT/RLHF); Rafailov et al., 2023 (DPO); documentação do Ollama e do Hugging Face TRL.